Lesson 10: ws:: API リファレンス

ws:: API Reference

StampFly Workshop

今日のゴール / Today's Goal

目標

ws:: API の全関数を把握し、独自ファームウェアで自由にセンサ・モータを操作する

- ws:: API 全 33 関数をカテゴリ別に理解
- 各 API の使い方をコード例で確認
- StampFlyState による上級アクセスを知る
- `sf app new` で独自プロジェクトを作成

ws:: API 概要 / API Overview

カテゴリ	概要	関数数
Motor Control	モータ制御・Arm/Disarm	7
RC Input	コントローラスティック入力	4
RC Buttons	ボタン・飛行モード	5
IMU Sensor	ジャイロ・加速度センサ	6
Estimation	ESKF 姿勢・高度推定	4
LED	LED 色制御	3
Utility	時刻・電圧・通信・出力	4
合計		**33**

`#include "workshop_api.hpp"` で全関数を使用可能。全関数は `ws::` 名前空間に属する。

より深いアクセスには **StampFlyState** を使用（後半で解説）。

Motor Control API（1/2）：モータ制御

関数	引数	説明
`motor_set_duty(id, duty)`	id: 1–4, duty: 0.0–1.0	個別モータのデューティ設定
`motor_set_all(duty)`	duty: 0.0–1.0	全モータを同一デューティに
`motor_stop_all()`	—	全モータ即時停止
`motor_mixer(t, r, p, y)`	各 float	スラスト＋姿勢ミキシング出力

```
// Individual motor control
ws::motor_set_duty(1, 0.3f);  // Motor 1 (FR) = 30%
ws::motor_stop_all();          // Emergency stop

// Mixer: thrust + attitude corrections
ws::motor_mixer(0.5f, 0.0f, 0.0f, 0.0f);  // Hover
```

Motor Control API（2/2）: Arm / Disarm

関数	戻り値	説明
`arm()`	void	モータ出力を有効化（Arm）
`disarm()`	void	モータ出力を無効化（Disarm）
`is_armed()`	bool	Arm 状態を返す（`true` = Armed）

```
if (ws::rc_throttle() < 0.05f) ws::arm();

if (ws::is_armed()) {
    ws::motor_mixer(ws::rc_throttle(),
        ws::rc_roll(), ws::rc_pitch(), ws::rc_yaw());
} else {
    ws::motor_stop_all();
}
```

注意: `arm()` 前はモータコマンドを送っても回転しない

RC Input API: スティック入力

関数	範囲	説明
rc_throttle()	$[0.0, 1.0]$	スロットル（上がプラス）
rc_roll()	$[-1.0, 1.0]$	ロール（右がプラス）
rc_pitch()	$[-1.0, 1.0]$	ピッチ（前がプラス）
rc_yaw()	$[-1.0, 1.0]$	ヨー（右回転がプラス）

```
void loop_400Hz(float dt) {
    float thr = ws::rc_throttle();  // 0.0 - 1.0
    float rol = ws::rc_roll();      // -1.0 - 1.0
    float pit = ws::rc_pitch();
    float yaw = ws::rc_yaw();
    ws::motor_mixer(thr, rol, pit, yaw);  // ACRO mode
}
```

RC Buttons / Mode API

関数	戻り値	説明
rc_throttle_yaw_button()	bool	左スティック押し込みボタン
rc_roll_pitch_button()	bool	右スティック押し込みボタン
rc_stabilize_acro_mode()	bool	ACRO モード判定（true = ACRO）
rc_alt_mode()	bool	高度保持モード判定
rc_pos_mode()	bool	位置保持モード判定

```
void loop_400Hz(float dt) {
    if (ws::rc_stabilize_acro_mode()) {
        acro_control(dt);        // ACRO: rate control
    } else {
        stabilize_control(dt);   // Stabilize: angle control
    }
}
```

IMU Sensor API

関数	単位	説明
gyro_x(), gyro_y(), gyro_z()	rad/s	ロール/ピッチ/ヨー軸角速度（BMI270）
accel_x(), accel_y(), accel_z()	m/s^2	前方/右方/下方加速度（NED 座標系）

```
void loop_400Hz(float dt) {
    float gx = ws::gyro_x();     // [rad/s] roll rate
    float gy = ws::gyro_y();     // [rad/s] pitch rate
    float gz = ws::gyro_z();     // [rad/s] yaw rate
    float az = ws::accel_z();    // [m/s^2] ~9.81 at rest

    ws::print(">gyro_x:%.3f", gx);
    ws::print(">accel_z:%.2f", az);
}
```

静止時: accel_z() $\approx$ 9.81（NED 座標系、下向き正）

Estimation API: ESKF 推定値

関数	単位	説明
estimated_roll()	rad	ESKF ロール推定角
estimated_pitch()	rad	ESKF ピッチ推定角
estimated_yaw()	rad	ESKF ヨー推定角
estimated_altitude()	m	ESKF 推定高度（正 = 上）

```
void loop_400Hz(float dt) {
    float roll  = ws::estimated_roll();      // [rad]
    float pitch = ws::estimated_pitch();
    float alt   = ws::estimated_altitude(); // [m]
    // P control: level the drone
    ws::motor_mixer(ws::rc_throttle(),
        (0.0f-roll)*0.5f, (0.0f-pitch)*0.5f, ws::rc_yaw());
}
```

ESKF はジャイロ+加速度+気圧+ToF+磁気+光学フローを統合（L09 参照）

LED Control API

関数	引数/戻り値	説明
`led_color(r, g, b)`	r,g,b: 0–255 (uint8_t)	LED 色を RGB で設定
`disable_led_task()`	void	システム LED タスクを停止
`is_led_task_disabled()`	bool	LED タスクが停止中か確認

```
void setup() {
    ws::disable_led_task();  // Take over LED control
}
void loop_400Hz(float dt) {
    float v = ws::battery_voltage();
    if (v > 3.8f)       ws::led_color(0, 255, 0);     // Green
    else if (v > 3.5f) ws::led_color(255, 165, 0);  // Orange
    else                 ws::led_color(255, 0, 0);     // Red
}
```

注意: `disable_led_task()` を呼ばないとシステムが LED を上書きする

Utility API

関数	戻り値	説明
`millis()`	uint32_t (ms)	起動からの経過時間
`battery_voltage()`	float (V)	バッテリー電圧
`print(fmt, ...)`	void	printf 形式でデバッグ出力
`set_channel(ch)`	void	WiFi チャネル設定（1, 6, 11）

```
void setup() {
    ws::set_channel(6);
    ws::print("Lesson 10: API Reference");
}
void loop_400Hz(float dt) {
    static uint32_t last = 0;
    uint32_t now = ws::millis();
    if (now - last >= 20) {  // 50 Hz decimation
        last = now;
        ws::print(">battery:%.2f", ws::battery_voltage());
    }
}
```

StampFlyState: 全センサ + 推定値

メソッド	ソース	取得データ
`getIMUData(a, g)`	BMI270 IMU	加速度 + ジャイロ生値
`getIMUCorrected(a, g)`	バイアス補正	バイアス除去済み IMU
`getBaroData(alt, p)`	BMP280 気圧	高度 [m], 気圧 [Pa]
`getMagData(mag)`	BMM150 磁気	磁気 x,y,z [μT]
`getToFData(b, f)`	VL53L3CX ToF	下方/前方距離 [m]
`getFlowData(vx, vy)`	PMW3901 フロー	速度 vx,vy [m/s]
`getFlowRawData(...)`	PMW3901 生値	dx,dy [counts], squal
`getAttitude()`	ESKF 推定	roll, pitch, yaw [rad]
`getPosition()`	ESKF 推定	x, y, z [m]
`getVelocity()`	ESKF 推定	vx, vy, vz [m/s]
`getGyroBias()`	ESKF 推定	バイアス [rad/s]
`getAccelBias()`	ESKF 推定	バイアス [m/s^2]

ハードウェアセンサ仕様 / Hardware Sensors

センサ	型番	サンプルレート	測定量
IMU	BMI270	400 Hz	加速度 + ジャイロ
気圧	BMP280	50 Hz	気圧 → 高度
磁気	BMM150	10–30 Hz	磁気ベクトル
ToF（下方）	VL53L3CX	30 Hz	対地距離（0–2 m）
ToF（前方）	VL53L3CX	30 Hz	前方距離（0–2 m）
光学フロー	PMW3901	100 Hz	対地速度

ws:: API は IMU のみ直接アクセス可（`gyro_x()`, `accel_x()` 等）。
気圧・磁気・ToF・光学フローは **StampFlyState** 経由で取得。

ESKF は全センサを統合して姿勢・位置・速度を推定（Lesson 9 参照）。

エコシステム概要 / Ecosystem Overview

開発ツール

- `sf build / flash / monitor`
- `sf cal gyro/accel/mag`
- `sf log wifi / capture / viz`
- `sf sim list / run`

解析・応用

- Python SDK（開発中）
- Jupyter Notebooks
- Genesis シミュレータ
- フライトログ解析

ワークショップで学んだスキルは、エコシステムの全ツールで活用可能

独自ファーム作成: sf app new

sf app new でカスタムプロジェクトを生成

vehicle のコンポーネントを再利用しつつ、独自の main.cpp を記述

```
# Create project
sf app new my_drone
# -> firmware/my_drone/ が生成される

# Build and flash
sf build my_drone
sf flash my_drone -m
```

仕組み: EXTRA_COMPONENT_DIRS で vehicle/components と common を参照。

IMU, 気圧, ToF, 光学フロー, モーター等すべてのコンポーネントが使える。

Teleplot によるセンサ可視化

```
auto& s = StampFlyState::getInstance();
Vec3 accel, gyro;
s.getIMUData(accel, gyro);
float baro_alt, pressure;
s.getBaroData(baro_alt, pressure);
float tof_bottom, tof_front;
s.getToFData(tof_bottom, tof_front);
auto att = s.getAttitude();     // ESKF roll,pitch,yaw
float alt = ws::estimated_altitude();

// Teleplot output (>name:value format)
ws::print(">baro_alt:%.2f", baro_alt);
ws::print(">tof_bottom:%.3f", tof_bottom);
ws::print(">eskf_alt:%.2f", alt);
ws::print(">eskf_roll:%.1f", att.x * 57.3f);
```

気圧高度 vs ToF vs ESKF 推定高度の比較で、センサ融合の効果を観察

実習: 全センサ読み取り / Hands-on

```
#include "workshop_api.hpp"
#include "stampfly_state.hpp"
void setup() { ws::print("Lesson 10: API Reference"); }
void loop_400Hz(float dt) {
    static uint32_t tick = 0; tick++;
    if (tick % 8 != 0) return;  // 50 Hz decimation
    // ws:: API for ESKF estimation
    ws::print(">eskf_roll:%.1f", ws::estimated_roll()*57.3f);
    ws::print(">eskf_alt:%.2f", ws::estimated_altitude());
    // StampFlyState for raw sensor data
    auto& s = StampFlyState::getInstance();
    float baro_alt, pressure;
    s.getBaroData(baro_alt, pressure);
    ws::print(">baro_alt:%.2f", baro_alt);
}
```

チェックポイント / Checkpoint

確認事項

- ☐ ws:: API 全 33 関数のカテゴリと役割を把握した
- ☐ Motor / RC / IMU / Estimation の使い方を理解した
- ☐ StampFlyState で全センサ生値を取得できた
- ☐ Teleplot で複数センサのグラフを同時表示した
- ☐ `sf app new` でカスタムプロジェクトを作成できた

次のレッスン

Lesson 11: Python SDK プログラム飛行